

Equation Reference

Every formula MICKSUS uses, for checking its numbers by hand. 82 equations · v1.28.0 · in-app copy at Help > Equations

Units. Geometry is millimetres in a right-handed frame — +X forward, +Y left, +Z up, origin at ground on the centreline, left corner modelled and mirrored. The dynamics layer is inches, pounds and seconds, so **mass = $W / 386.4$** (g in in/s²). Using 32.2 is the classic factor-of-12 error.

Symbols

Symbol	Meaning	Units
W	total vehicle weight incl. driver	lb
W_F, W_R	front / rear AXLE weight (with driver)	lb
W_u	unsprung weight (per side unless noted)	lb
W_s	sprung weight = W - total unsprung	lb
L	wheelbase	in
t	track width (t_f front, t_r rear)	in
a, b	sprung CG to FRONT / REAR axle (a + b = L)	in
h	CG height above ground	in
h_s	SPRUNG-mass CG height above ground	in
H	roll moment arm = h_s - roll-axis height at the CG	in
z_RC	roll-centre height above ground	in
r_l	loaded tire radius	in
K_s	spring rate (at the spring)	lb/in
K_w	wheel rate = K_s x MR ²	lb/in
K_t	tire vertical rate	lb/in
K_r	ride rate = K_w in series with K_t	lb/in
K_phi	axle roll rate	lb-ft/deg
MR	motion ratio = SPRING (shock) travel / wheel travel; < 1	-
c	damper coefficient (c_w = at the wheel)	lb-s/in
zeta	damping ratio c / c_crit	-
A_y, A_x	lateral / longitudinal acceleration (+A_x = braking)	g
m_s, m_u	sprung / unsprung MASS = weight / 386.4	lb-s ² /in
g	gravity — 386.4 in/s ² in this tool's imperial units	in/s ²
f	frequency	Hz
omega	angular frequency = 2*pi*f	rad/s
s	Laplace variable; s = j*omega for frequency response	-
mu	peak friction coefficient	-
C_alpha	cornering stiffness (per axle unless noted)	lb/deg
T	axial TENSION in a two-force member (+ = pulled)	lb
n_hat	unit vector along a member, inboard -> outboard	-

1 · Conventions

Internal geometry is millimetres in a right-handed frame: +X forward, +Y left, +Z up, origin at ground level on the centreline. The tool models the LEFT corner and mirrors it. Display and CAD export default to the CHASSIS convention (+Y forward). The dynamics layer is imperial: inches, pounds, seconds.

Mass from weight

$$m = W / g, \quad g = 386.4 \text{ in/s}^2$$

where W lb; m lb-s²/in

Using 32.2 instead of 386.4 is the classic factor-of-12 error. $386.4 = 32.2 \text{ ft/s}^2 \times 12 \text{ in/ft}$.

dynamics.py, impact.py, ride_freq.py

Angle sign conventions

camber < 0 = top of tire leans toward the centreline; toe > 0 = toe-IN; caster > 0 = kingpin top rearward;

KPI > 0 = kingpin top inboard

Matches SAE J670.

metrics.py

2 · Kinematic solver core

The double-wishbone corner has one degree of freedom once the tie rod is fixed. It is solved in closed form — no iteration except a 1-D root find for the requested wheel travel.

Arm rotation (Rodrigues)

$$\mathbf{p}' = \mathbf{o} + \mathbf{r} \cos(t) + (\mathbf{a_hat} \times \mathbf{r}) \sin(t) + \mathbf{a_hat} (\mathbf{a_hat} \cdot \mathbf{r}) (1 - \cos(t)), \quad \mathbf{r} = \mathbf{p} - \mathbf{o}$$

where $\mathbf{o}$ point on the pivot axis; $\mathbf{a_hat}$ unit axis; t rotation angle

Rotates a ball joint about its arm's chassis pivot axis.

solver.py

Upper ball joint: circle $\cap$ sphere

$$A \cos(b) + B \sin(b) = C, \quad b = \phi \pm \delta$$

where $A = 2^*w.u$; $B = 2^*w.v$; $C = l^2 - w.w - u.u$; $w = c - LBJ$; u radius vector; $v = \text{axis} \times u$; $l = \text{kingpin length}$; $\phi = \text{atan2}(B, A)$; $\delta = \arccos(C / \sqrt{A^2 + B^2})$

The UBJ lies on the circle swept by the upper arm AND on the sphere of fixed kingpin length about the LBJ. The two roots are the two ASSEMBLY BRANCHES; the branch is fixed once from the static pose and never allowed to switch.

solver.py

Tie-rod outer: trilateration

$$x = (r1^2 - r2^2 + d^2) / (2d); \quad y = (r1^2 - r3^2 + i^2 + j^2) / (2j) - (i/j)*x; \quad z = \pm \sqrt{r1^2 - x^2 - y^2}$$

where $r1, r2, r3$ distances from LBJ, UBJ, tie-rod inner; $d = |UBJ - LBJ|$; i, j local-frame coordinates of the third centre

Intersection of three spheres. The $\pm$ sign is the mirror solution, fixed from the static geometry.

solver.py

Upright pose (Kabsch)

$$\mathbf{H} = \mathbf{A}^T \mathbf{B}; \quad \mathbf{U} \mathbf{S} \mathbf{V}^T = \text{svd}(\mathbf{H}); \quad \mathbf{R} = \mathbf{V} * \text{diag}(1, 1, \det(\mathbf{V} \mathbf{U}^T)) * \mathbf{U}^T$$

where $\mathbf{A}$ static upright points (centred); $\mathbf{B}$ current points (centred)

Exact here, not a fit: three rigid points fully define the pose. Carries the wheel centre and spindle axis along.

solver.py

Contact patch

$$\mathbf{CP} = \mathbf{WC} + \mathbf{R_tire} * \mathbf{d_hat}, \quad \mathbf{d_hat} = \text{unit}(\mathbf{s_hat} \times \text{unit}(\mathbf{s_hat} \times \mathbf{z_hat}))$$

where $\mathbf{WC}$ wheel centre; $\mathbf{s_hat}$ spindle axis; $\mathbf{R_tire}$ tire radius

Goes one tire radius 'down' WITHIN the wheel plane, so camber tips the patch sideways as on a real wheel.

solver.py

3 · Wheel orientation angles

All read off the solved "spindle axis" $\mathbf{s_hat}$ and the kingpin vector $\mathbf{k} = \mathbf{UBJ} - \mathbf{LBJ}$.

Camber

$$\gamma = -\text{atan2}(s_z, s_y)$$

where $\mathbf{s_hat}$ spindle unit vector (points outboard, +Y on the left corner)

Negative = top of the tire toward the centreline.

metrics.py

Toe

$$\tau = -\text{atan2}(f_y, f_x), \quad \mathbf{f_hat} = \text{unit}(\mathbf{s_hat} \times \mathbf{z_hat})$$

where $\mathbf{f_hat}$ wheel heading in top view

Positive = toe-in.

metrics.py

Caster

$$\text{caster} = \text{atan2}(-k_x, k_z)$$

where $\mathbf{k} = \mathbf{UBJ} - \mathbf{LBJ}$

Positive = kingpin top rearward.

metrics.py

Kingpin inclination

$$\text{KPI} = \text{atan2}(-k_y, k_z)$$

where $\mathbf{k}$ as above

Positive = kingpin top inboard (left corner).

metrics.py

Kingpin ground intercept

$$\mathbf{P} = \mathbf{LBJ} + [(\mathbf{CP_z} - \mathbf{LBJ_z}) / k_z] * \mathbf{k}$$

where $\mathbf{P}$ where the steering axis pierces the ground plane through the contact patch

Uses the patch's own z , not $z = 0$ — the CAD-aligned origin puts ground below zero.

metrics.py

Scrub radius

$$r_{\text{scrub}} = \mathbf{CP_y} - \mathbf{P_y}$$

Positive = contact patch outboard of the steering axis.

metrics.py

Mechanical (caster) trail

$\text{trail} = P_x - CP_x$

Positive = patch behind the steering axis (stabilising).

metrics.py

4 · Instant centre & roll centre

Built in the **FRONT VIEW** — the transverse vertical plane (Y-Z), per SAE J670 — NOT the tilted 2-D manufacturing sketch plane. Each arm is sliced at its OWN ball-joint station.

Arm-plane normal

$n = (\text{inner}_r - \text{inner}_f) \times (\text{outer} - \text{inner}_f)$

where inner_f, inner_r the two chassis bushings; outer the ball joint

The ball joint's velocity is exactly parallel to n, which is what makes the construction exact for a spatial arm.

metrics.py

Front-view arm line

$n_y * y + n_z * z = n_y * BJ_y + n_z * BJ_z$

Slicing BOTH arms at the shared wheel-centre plane instead offsets each line by $n_x(BJ_x - x_{wc})$ — zero only for flat arms, tens of mm out at large kickup.

metrics.py

Side-view arm line

$n_x * x + n_z * z = n_x * BJ_x + n_z * BJ_z$

Same construction in the X-Z plane; governs wheel recession and anti-geometry.

metrics.py

Instant centre

$IC = \text{solve}([a1 \ b1; a2 \ b2] [y; z] = [c1; c2])$

where the two arm lines $a*y + b*z = c$

Parallel lines ($\det = 0$) means the IC is at infinity — the DESIGNED state for this tool's seeds.

metrics.py

Roll-centre height

$z_{RC} = (0 - CP_y) * (z_{IC} - CP_z) / (y_{IC} - CP_y)$

where CP contact patch; IC front-view instant centre

Height above the GROUND PLANE. The line runs contact patch -> IC, extended to the centreline $y = 0$.

metrics.py

5 · Motion ratio, rates and anti-geometry

Motion ratio

$MR = d(\text{shock length}) / d(\text{wheel travel})$

where $MR < 1$ for a shock mounted inboard on the arm

STANDARD convention (shock/wheel), so it feeds $K_w = K_s * MR^2$ with NO inversion. Computed by numerical gradient over the travel sweep, so it VARIES through travel. Before v1.28 this tool reported the reciprocal (wheel/shock, ~1.85); reading that into a textbook formula without inverting gives a 3.4x error, which is why it was changed.

metrics.py

Wheel rate

$K_w = K_s * MR^2$

where MR = shock travel / wheel travel, BELOW 1

Squared — the most common hand-calc slip. Damping goes the same way: $c_w = c * MR_{\text{damper}}^2$. Since $MR < 1$ the wheel rate is always LOWER than the spring rate; if your answer comes out higher, the ratio is upside down.

dynamics.py

Ride rate

$K_r = K_w * K_t / (K_w + K_t)$

Spring and tire in SERIES. Sprung-mass modes use K_r , never the bare wheel rate.

dynamics.py

Spring stacks

Series: $K = K1 * K2 / (K1 + K2)$; Parallel: $K = K1 + K2$

dynamics.py

Anti-dive (front, braking)

$\%anti = 100 * f_b * \tan(\phi_f) * L / h$

where f_b front brake fraction; ϕ_f support angle from the CONTACT PATCH to the side-view IC

Outboard brakes -> contact-patch line.

vehicle.py

Anti-lift (rear, braking)

$\%anti = 100 * (1 - f_b) * \tan(\phi_r) * L / h$

vehicle.py

Anti-squat (rear, drive)

$$\%anti = 100 * \tan(\theta) * L / h$$

where θ support angle from the WHEEL CENTRE to the side-view IC

Inboard (chassis-mounted) drive -> wheel-centre line. It differs from the dive/lift line by the loaded radius over the SVSA length, so one slope cannot serve both.
vehicle.py

6 · Mass, CG and the roll axis

CG longitudinal position

$$b = W_F * L / W; a = L - b$$

where b = CG to REAR axle; a = CG to FRONT axle

dynamics.py

Sprung weight

$$W_s = W - 2*W_{uf} - 2*W_{ur}$$

where W_{uf} , W_{ur} unsprung PER SIDE

dynamics.py

Sprung CG height

$$h_s = (W*h - 2*W_{uf}*r_{lf} - 2*W_{ur}*r_{lr}) / W_s$$

The unsprung mass sits at wheel-centre height, so removing it raises the remaining CG.

dynamics.py

Front sprung fraction

$$a_s = (W_F - 2*W_{uf}) / W_s$$

dynamics.py

Roll-axis height at the sprung CG

$$z_{RA} = z_{RCf} + (z_{RCr} - z_{RCf}) * (1 - a_s)$$

Linear interpolation along the roll axis.

dynamics.py

Roll moment arm

$$H = h_s - z_{RA}$$

THE handling lever. Shrink it and body roll falls.

dynamics.py

7 · Roll rates and roll gradient

Milliken RCVD ch. 16.3. Note the unit juggling: track and arm lengths in FEET inside these.

Anti-roll-bar contribution

$$K_{ARB} = K_{bar} * (IR * t / L_{arm})^2$$

where K_{bar} bar torsional rate lb-ft/deg; IR install ratio; t track ft; L_{arm} bar lever arm ft

dynamics.py

Axle roll rate (springs + bar, in series with tires)

$$K_{spring} = 12 * K_w * t_{ft}^2 / 2 \text{ [lb-ft/rad]}; K_{axle} = \text{series}(K_{ARB} + K_{spring}, K_{tire})$$

The tire is a series spring in roll too — ignoring it overstates roll stiffness.

dynamics.py

Roll gradient

$$d(\phi)/d(A_y) = -W_s * (H/12) / (K_{\phi_f} + K_{\phi_r})$$

where H in inches, /12 to feet; result deg/g

Negative = the body leans OUT of the turn (normal).

dynamics.py

8 · Load transfer

Milliken 18.4. The lateral transfer has THREE reactions per axle, not one: the sprung mass reacts elastically through the roll stiffness AND geometrically through the links at the roll centre, and the unsprung mass reacts directly at the wheel centre. Both sprung terms carry W_s — using total W and dropping the unsprung term overstates the transfer by roughly 12%.

Lateral transfer, front axle

$$dW_f = (A_y / t_f) * [W_s * H * K_{\phi_f} / (K_{\phi_f} + K_{\phi_r}) + W_s * (b/L) * z_{RCf} + W_{uf_axle} * r_{lf}]$$

where all lengths in FEET; W_{uf_axle} = 2 x per-side unsprung; t_f front track ft

Rear is the mirror with a/L and z_{RCr} .

dynamics.py

Longitudinal transfer

$$dW_x = A_x * W * h / L$$

Positive A_x = braking = load onto the front.

dynamics.py

Banked-road lateral demand

$A_{y_eff} = A_y \cos(\beta) - \sin(\beta)$
where β bank angle, POSITIVE = banked INTO the turn
Positive bank HELPS, hence the minus. SAE sign.
dynamics.py

Aerodynamic download

$F_z = 0.5 * \rho * V^2 * C_L * A$; ride drop = $F_{z_axle} / (2 * K_r)$
dynamics.py

Dynamic corner loads

$F_{out} = W_{axle}/2 + dW_{lat} \pm dW_x/2 + F_{aero}/2$; $F_{in} = W_{axle}/2 - dW_{lat} \pm dW_x/2 + F_{aero}/2$
where +dW_x/2 front, -dW_x/2 rear under braking
A negative result means that wheel has LIFTED.
dynamics.py

Lateral load transfer distribution

$TLLTD = dW_f / (dW_f + dW_r)$
Above the static front weight fraction = understeer trend.
dynamics.py

9 · Ride frequencies and damping

Corner ride frequency

$f_{ride} = (1/2\pi) * \sqrt{K_r * g / W_{sprung_corner}}$
Uses the RIDE rate (Spring + tire in series).
dynamics.py

Bounce frequency

$f_{bounce} = (1/2\pi) * \sqrt{(2 * K_{rf} + 2 * K_{rr}) / m_s}$
Decoupled approximation — the full-car model solves the coupled eigenproblem instead.
dynamics.py

Pitch radius of gyration (Olley)

$k_y = \sqrt{a * b}$
Dynamic index = 1, the flat-ride assumption. An ESTIMATE until you measure it.
dynamics.py

Pitch frequency

$f_{pitch} = (1/2\pi) * \sqrt{(2 * K_{rf} * a^2 + 2 * K_{rr} * b^2) / (m_s * a * b)}$
dynamics.py

Critical damping and damping ratio

$c_{crit} = 2 * \sqrt{K_w * m_{s_corner}}$; $\zeta = c_{wheel} / c_{crit}$, $c_{wheel} = c_{damper} * MR_d^2$
Racing practice is $\zeta \sim 0.3-0.5$ in bump.
dynamics.py

Olley ratio

$olley = f_{rear} / f_{front}$
Target 1.1-1.2 — rear stiffer so the car settles flat rather than pitching.
dynamics.py

10 · Steering and brakes

Understeer gradient

$K_{us} = W_F / C_{\alpha_F} - W_R / C_{\alpha_R}$
Positive = understeer. deg/g.
dynamics.py

Ackermann percentage

$\%Ack = 100 * (d_i - d_o) / (d_{ideal} - d_o)$; $\cot(d_o) - \cot(d_i) = t / L$
where d_i, d_o inner / outer wheel steer angles
100 = perfect Ackermann, 0 = parallel steer, negative = anti-Ackermann.
sweeps.py

Low-speed turn diameter

$D = 2 * \sqrt{(R_{rear} + t/2)^2 + L^2}$, $R_{rear} = L / \tan(\text{mean steer})$
sweeps.py

Brake chain

$F_{mc} = F_{pedal} * pedal_ratio * bias$; $P = F_{mc} / (\pi * (bore/2)^2)$; $clamp = P * A_{caliper}$; $T = 2 * (2 * \mu_{pad} * clamp * r_{eff})$; $F_{brake} = T / r_l$; $decel = \text{sum}(F_{brake}) / W$
where r_{eff} effective rotor radius to the pad centre
The 2 x (2 x ...) is two wheels per axle, two pads per caliper.
dynamics.py

11 · Tire model

All force-and-moment values for the SunF 23x7-10 are ESTIMATES — no measured data exists. Treat outputs as qualitative.

Saturating lateral force

```
F_y = mu * F_z * tanh( C_alpha / (mu * F_z) * alpha )
```

where alpha slip angle deg; C_alpha per-tire cornering stiffness

Matches the cornering stiffness at small slip and saturates at $\mu * F_z$. Stand-in for a fitted Pacejka curve.

tire.py

Cornering stiffness from load

```
C_alpha = CC * F_z
```

where CC normalised cornering coefficient, ~0.11 /deg for a knobby (street radials 0.15-0.25)

tire.py

Peak-slip marker

```
alpha_peak = (mu * F_z / C_alpha) * artanh(0.98)
```

Where the model reaches 98% of saturation.

tire.py

Pacejka seed coefficients

```
D = mu * F_z; C ~ 1.40; B = C_alpha_rad / (C * D); E ~ 0.90
```

Built from the estimated peak and initial slope — NOT a fit to data.

tire.py

Tire vertical rate vs pressure

```
K_t ~ 65 + 22 * psi [lb/in]
```

Informed estimate for a small bias tire; verify on your own tire.

tire.py

12 · Halfshaft / CV joints

Joint working angle

```
angle = arccos( |u_hat . v_hat| )
```

where u_hat shaft direction; v_hat the joint's reference axis (gearbox output for the inner, spindle for the outer)

Absolute value: the joint does not care about sign.

halfshaft.py

Plunge budget

```
plunge_in = total * pct/100; pull_out = total * (1 - pct/100)
```

Measured from the RIDE-HEIGHT shaft length, and asymmetric — ride height rarely sits centred in the joint's stroke.

halfshaft.py

13 · Impact loads (FEA export)

Impulse-momentum for the wheel load, then rigid-body statics for the load path. Quasi-static: the peak force is applied as a static load.

Impulse and average force

```
J = m * dv, m = W/g; F_avg = J / dt
```

dt is the CONTACT TIME. Halving it doubles every mount load, which is why measuring it matters.

impact.py

Design peak force

```
F_peak = F_avg * pulse * share * SF
```

where pulse: rectangular 1.00, half-sine $\pi/2 = 1.571$, triangular 2.00; share = fraction of the car this corner takes; SF safety factor

impact.py

Drop height to impact speed

```
v = sqrt(2 * g * height)
```

impact.py

Friction-circle combined load

```
|F_horizontal| = mu * F_z, swept in direction; M_z = -F_y * pneumatic trail
```

The worst member load is a COMBINED case, never one axis alone.

impact.py

Rigid-body equilibrium (per body)

```
sum(F) = 0; sum(M) = 0
```

Six equations per body, solved by least squares.

impact.py

Two-force member

```
force on the OUTBOARD body = -T * n_hat; force on the INBOARD (chassis) = +T * n_hat
```

where T tension (+ = pulled); n_hat = unit(outboard - inboard)

Getting this sign wrong breaks equilibrium silently.

impact.py

Control-arm reduced equation

$$a_hat \cdot \sum (r_i - p1) \times F_i = 0$$

where a_hat the arm's bushing axis; $p1$ one bushing; F_i every load on the arm (ball joint, shock, arm-mounted toe link)

Two spherical bushings ON A LINE produce no moment about that line, so dotting the moment equation with the axis annihilates both bushing unknowns and leaves exactly one scalar constraint. This is why the shock's position along the arm changes the answer.

impact.py

Indeterminate bushing split

$$\min ||R|| \text{ subject to equilibrium (min-norm least squares)}$$

Two spherical joints on a line cannot resolve force ALONG that line. The TOTAL axial is reported; size a bracket against all of it, not half.

impact.py

Beam internal loads

$$F_internal(s) = -\sum (F_i : station_i \leq s); M_internal(s) = -\sum (p_i - cut) \times F_i$$

Bending is an INTERNAL stress resultant — it never appears in a joint reaction, which is why it needs its own output.

impact.py

Rocker (bellcrank)

$$a_hat \cdot [(r_pr) \times F_pushrod + D \cdot (r_d) \times u_hat_d] = 0$$

where a_hat pivot axis; r measured from the pivot; D damper axial force

One equation for the damper force; force balance then gives the pivot bearing reaction.

impact.py

Virtual-work check

$$F_shock = F_wheel / MR$$

where $MR = \text{shock/wheel} < 1$, so the shock force EXCEEDS the wheel force

INDEPENDENT of the statics — it uses only the kinematic sweep. The single best check that a load path is right.

impact.py

14 · Quarter-car frequency response

The Bode plot. Sprung mass on the wheel-rate spring and damper, unsprung mass on the tire spring.

Equations of motion

$$m_s \ddot{z}_s = -K_w(z_s - z_u) - c(\dot{z}_s - \dot{z}_u); m_u \ddot{z}_u = +K_w(z_s - z_u) + c(\dot{z}_s - \dot{z}_u) - K_t(z_u - z_r)$$

where z_s body; z_u wheel; z_r road; $' = d/dt$

ride_freq.py

Frequency-response system

$$\begin{bmatrix} m_s s^2 + c s + K_w & -(c s + K_w) \\ -(c s + K_w) & m_u s^2 + c s + K_w + K_t \end{bmatrix} \begin{bmatrix} Z_s \\ Z_u \end{bmatrix} = \begin{bmatrix} 0 \\ K_t Z_r \end{bmatrix}$$

where $s = j\omega$

Invert this 2x2 and every plotted curve is a ratio to Z_r .

ride_freq.py

Body and wheel-hop frequencies

$$f_body \sim (1/2\pi) \sqrt{K_ride/m_s}; f_hop \sim (1/2\pi) \sqrt{(K_w + K_t)/m_u}$$

Approximations. The tool takes the exact values from the undamped eigenproblem, which is damping-independent — peak-picking a damped curve is unreliable near $\zeta = 1$.

ride_freq.py

Isolation crossover

$$\text{transmissibility} = 1 \text{ exactly at } f = \sqrt{2} \cdot f_body$$

Below it you amplify the road; above it you isolate — and above it MORE damping makes isolation WORSE.

ride_freq.py

Terrain excitation

$$f = V / \lambda$$

where V road speed; λ bump spacing

Converts a resonance into the speed that excites it.

ride_freq.py

Dynamic tire load / lift-off

$$dF_z = (\text{tire deflection ratio}) \cdot K_t \cdot \text{bump}; \text{tire lifts when } dF_z \geq \text{static corner load}$$

How a dimensionless curve becomes a decision.

ride_freq.py

15 · Full-car (7-DOF) ride model

Heave, roll and pitch of the sprung mass plus four unsprung masses.

Sprung displacement at a corner

```
z_i = z_s + phi*y_i - theta*x_i
```

where x_i, y_i corner position from the sprung CG (+x forward, +y left); phi roll; theta pitch

From the rotation vector: $dz = \phi y - \theta x$. So $+\theta = \text{NOSE DOWN}$ and $+\phi = \text{LEFT SIDE UP}$.

full_car.py

Assembled system

```
M*q'' + C*q' + K*q = F_road, q = [z_s, phi, theta, z_u1..z_u4]
```

A corner spring K adds $K*b*b^T$ to the sprung block with $b = [1, y_i, -x_i]$, $-K*b$ to the coupling, and K to the unsprung diagonal.

full_car.py

Anti-roll bar stiffness

K adds $k_{\text{bar}} * g * g^T$, $g = b_{\text{left}} - b_{\text{right}}$; $k_{\text{bar}} = K_{\text{phi_ARB}} / \text{track}^2$

where K_{phi} in lb-in/rad = lb-ft/deg $\times 12 \times 180/\pi$

Note track^2 , NOT $\text{track}^2/2$ — the $/2$ in the spring term comes from each wheel moving only half the track.

full_car.py

Wheelbase filtering

```
Z_rear = Z_front * exp(-j*omega*tau), tau = L / v
```

The rear axle meets each bump L/v after the front, so the response depends on SPEED.

full_car.py

Comb nulls

pitch null at $f = n*v/L$ (axles in phase); heave null at $f = (2n+1)*v/(2L)$ (axles opposed)

Exact for a symmetric car; deep minima on a real one.

full_car.py

Modal frequencies and labelling

```
solve det(K - omega^2 M) = 0; label by KE share = m_i * v_i^2 / sum
```

Seven bare frequencies are useless without knowing which is bounce, pitch, roll or hop.

full_car.py

Froude scaling (model cars)

length $\sim \lambda$; time $\sim \sqrt{\lambda}$; frequency $\sim 1/\sqrt{\lambda}$; velocity $\sim \sqrt{\lambda}$

where λ scale factor (0.1 for 1:10)

Gravity does not scale. At 1:10 frequencies are 3.16x higher and speeds 3.16x lower for dynamic similarity.

full_car.py